Lesson4_2_1

September 5, 2026

1 Introduction

This web site is running a "Jupyter notebook," which allows you to execute interactive Octave code directly in your browser. Each (part of a code) line preceeded by a percent symbol "%" is a code comment.

To execute any Octave commands, first enter the commands in a code cell, and then depress the "shift" and "enter" keys on your keyboard at the same time, making sure that your cursor is placed inside of the code cell. I refer to this as "< shift >< enter >" in my instructions below.

The output from this notebook will be used to provide answers for this lesson's practice quiz. As you proceed through the specialization, you will use more and more Jupyter notebooks to write Octave code to implement Kalman-filter algorithms.

[2]:
```
% Place your cursor in this input code cell. Then, type < shift >< enter > to␣
↪execute

rms = @(x) sqrt(mean((x).^2)); % define root-mean-squared function
```

1.0.1 Implement the Monte-Carlo method to approximate the integral in the lesson

Note that the default values of N and sx are 10000 and 0.75. You will change these in the quizzes but may need to change them back to their default values at some point.

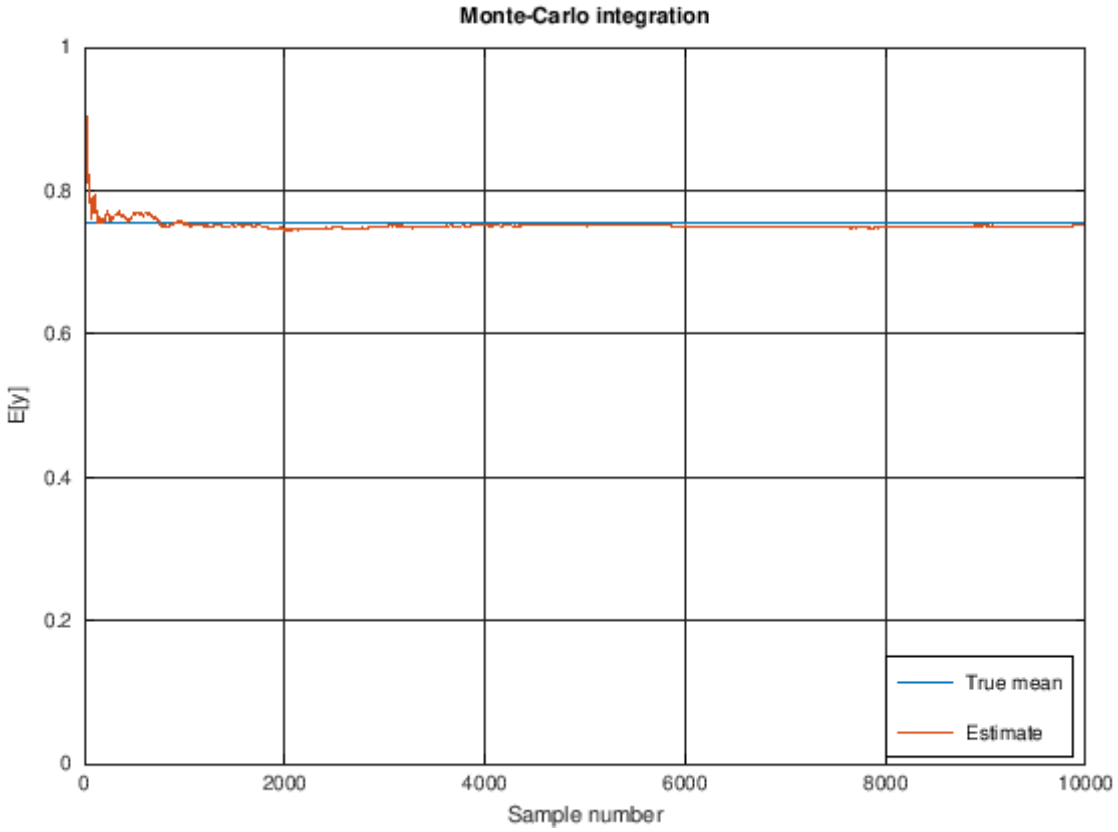
[3]:
```
N = 10000; sx = 0.75;      % problem constants

trueMean = exp(-sx^2/2);  % Found by closed-form integration techniques

randn("state",0);
x = sx*randn(1,N);         % generate sample pts.
y = cos(x);                % function evaluations
muEst = cumsum(y)./(1:N); % running approx. using Monte-Carlo method
```

1.0.2 Plot results

[4]:
```
plot([0 N-1],trueMean*[1 1],0:N-1,muEst);
legend('True mean','Estimate','location','southeast');
xlabel('Sample number');
ylabel('E[y]'); ylim([0 1]); grid on
title('Monte-Carlo integration');
```



[5]:
```
rmse = rms(trueMean-muEst(end-1999:end));
fprintf('RMSE over final 2000 iterations is: %g\n',rmse)
```

```
RMSE over final 2000 iterations is: 0.00488066
```

[ ]: